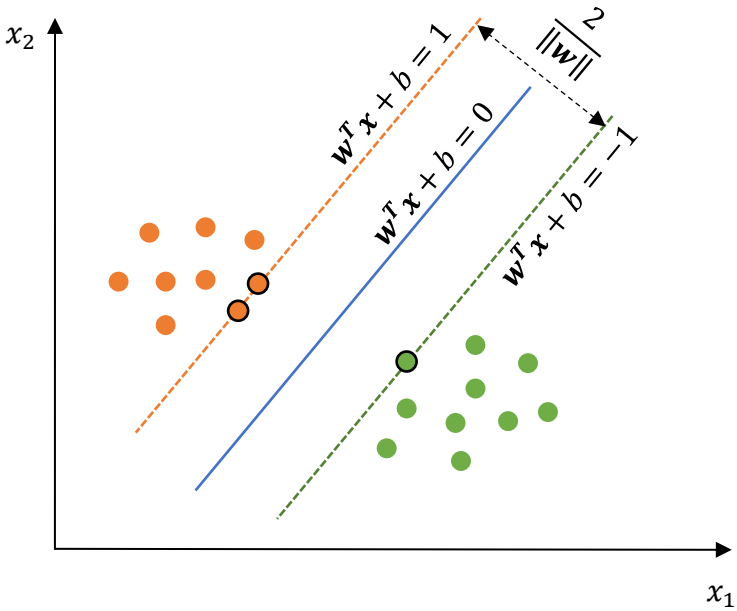
CG3201 Machine Learning and Deep Learning
Lab 1 – Support Vector Machine

1. Introduction



Support Vector Machine (SVM) is a supervised learning algorithm that finds an optimal hyperplane, $\boldsymbol{w}^T\boldsymbol{x}+b=0$, to separate data points belonging to different classes. Its objective is to maximize the margin between the hyperplane and the closest data points from each class, known as support vectors. In the above figure, the support vectors are the data points outlined in black.

The lines passing by the support vectors are given by $\boldsymbol{w}^T\boldsymbol{x}+b=1$ (the orange dashed line) and $\boldsymbol{w}^T\boldsymbol{x}+b=-1$ (the green dashed line). Ideally, all data points should lie outside the region between these two lines. That is:

- For the positive class: $y_i=1$, $\boldsymbol{w}^T\boldsymbol{x_i}+b\geq 1$
- For the negative class: $y_i=-1$, $\boldsymbol{w}^T\boldsymbol{x_i}+b\leq -1$

Equivalently, it can be written as $y_i(\boldsymbol{w}^T\boldsymbol{x_i}+b)\geq 1$ for all data points $\boldsymbol{x_i}$.

The margin between the orange dashed line and the green dashed line, which we aim to maximize, is given as $\frac{2}{\|\boldsymbol{w}\|}$. In this lab, we find the decision boundary that maximize this margin subject to the constraint $y_i(\boldsymbol{w}^T\boldsymbol{x_i}+b)\geq 1$ for all i. This can be achieved by minimizing the loss function:

$$L=\frac{1}{2}\|\boldsymbol{w}\|+C\sum_i \max\left(0,1-y_i(\boldsymbol{w}^T\boldsymbol{x_i}+b)\right) \tag{1}$$

$$L = \frac{1}{2}\boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{w} + C\sum_i \max\left(0, 1 - y_i(\boldsymbol{w}^T\boldsymbol{x_i} + b)\right) \tag{2}$$

This optimization problem can be solved using Stochastic Gradient Descend (SGD):

$$\boldsymbol{w} = \boldsymbol{w} - \eta\frac{\partial L}{\partial \boldsymbol{w}} \tag{3}$$

$$b = b - \eta\frac{\partial L}{\partial b} \tag{4}$$

Where η is the learning rate.

Derive the update formulas for $\boldsymbol{w}$ and b in gradient descent by considering the loss function L in two cases: when $y_i(\boldsymbol{w}^T\boldsymbol{x_i} + b) \geq 1$, and when this condition is violated.

- If $y_i(\boldsymbol{w}^T\boldsymbol{x_i} + b) \geq 1$: $L =$? hinge is 0 since (1-yi...), so loss = 1/2 ||w||^2 hence gradients are dL/dw = w and dL/db = 0
- If $y_i(\boldsymbol{w}^T\boldsymbol{x_i} + b) < 1$: $L =$? loss=1/2||w||^2 + C(1-y(wTx+b)) gradients: dL/dw = w-Cyx, dL/db = -Cy

As you read the introduction, consider: if you remove a data point that is not a support vector (i.e., it's far from the boundary), would the hyperplane change? What if you remove a data point that is a support vector? This intuition is key for the later quiz :)

Points with y(wTx+b)>1contribute zero hinge loss/zero hinge gradient, so removing them usually doesn't move the boundary much. Points with mi≤1 (on/inside the margin or misclassified) do drive updates; removing them can change the hyperplane

2. Dataset

In this lab, we use the UCI wine quality dataset. Each sample contains 11 physicochemical features (such as citric acid, sulfur dioxide, pH, and sulphates) and is labeled with a wine quality score ranging from 3 to 9. For the binary classification task, these scores are mapped into two classes: labels ≤ 5 are considered low quality, while labels > 5 are considered high quality. For the multi-class classification task, the original labels are used without modification.

Data loading and preprocessing are provided in the template via the load_data() function. The dataset is split into a training set of 4,800 samples and a test set containing the remaining samples. For the binary classification task, set the argument binary=True. For the multi-class classification task, set it to False.

3. Binary Classification with SVM

a) Implement an SVM class to perform the binary classification task. A template is provided with three methods: __init__(<hyperparameters>) to initialize the hyperparameters such as regularization parameter C, learning rate η, and max gradient descend iteration; fit(X, y) to train the SVM; and predict(X) to predict labels (-1 or $+1$).

.fit(): SGD on hinge loss

Binary classification
Accuracy on training set: 0.634375
Accuracy on test set: 0.6299351797289334

Since the difference is small, we are not overfitting, but it is possible that the model is not complex enough (possible underfitting), especially since the accuracy is low at 0.63. We will need to tune the hyper parameters to increase accuracy.

b) Apply the SVM classifier to the provided dataset and evaluate the accuracy on both the training set and the test set.

4. Hyperparameter Tuning and Observations

a) Try different values of C and η and observe the changes in accuracy.

b) Plot the training and test accuracy for C = 0.001, 0.01, 0.1, 1, 10, and 100; $\eta = 0.0001$; and maximum learning iterations of gradient decent is 1000. Use log scale for the horizontal axis by setting plt.xscale('log'). Could you explain why we use the log scale?

c) Plot the margin, $\frac{2}{\|\boldsymbol{w}\|}$, for C = 0.001, 0.01, 0.1, 1, 10, and 100; $\eta = 0.0001$; and maximum learning iterations of gradient decent is 1000. Use log scale for the horizontal axis. Observe how the margin changes when C is increased or decreased. How does it affect the accuracy?

Hint: Paying attention to how C influences the model's behavior is helpful for the later quiz. :)

5. Multi-Class Classification with SVM

a) Implement a MultiClass SVM to perform multi-class classification using the one v.s. all approach. A template is provided with three methods: __init__(<hyperparameters>) to initialize the hyperparameters, fit(X, y) to train the SVM, and predict(X) to predict labels.

Hint:

- During training, construct one binary SVM per class, where each classifier learns to distinguish samples of that class from all others (assigning label 1 to the target class and -1 to the rest).

- During prediction, apply all classifiers to a sample and assign the label corresponding to the classifier that produces the highest score $\boldsymbol{z_i} = \boldsymbol{w}^T \boldsymbol{x_i} + b$.

b) Apply the SVM classifier to the provided dataset and evaluate the accuracy on both the training set and the test set.

c) In evaluating a classification task, a confusion matrix is used to summarize how samples from each class are predicted across different classes. Rows correspond to actual labels, and columns correspond to predicted labels. For example, in a classification problem where the labels are wine quality scores in {3, 4, 5, 6, 7, 8, 9}, the confusion matrix has the following form:

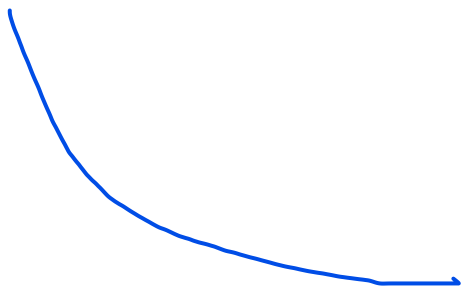
	Predicted 3	Predicted 4	Predicted 5	Predicted 6	Predicted 7	Predicted 8	Predicted 9
Actual 3	15	5	0	0	0	0	0

Actual 4	3	40	10	2	0	0	0
Actual 5	0	8	150	20	5	0	0
Actual 6	0	2	30	200	15	3	0
Actual 7	0	0	5	25	80	10	2
Actual 8	0	0	0	5	12	35	3
Actual 9	0	0	0	0	1	2	10

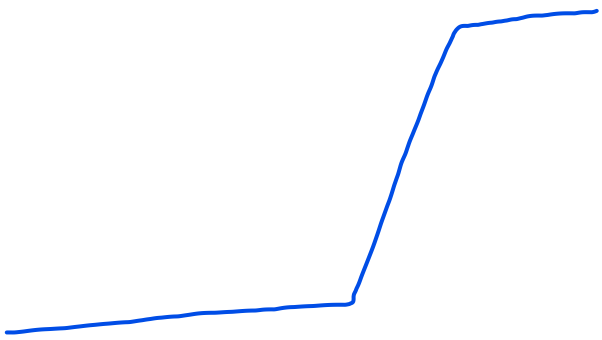
Print the confusion matrix on the test data after performing multi-class classification using SVM.

Note: The confusion matrix shown above is for illustration and does not reflect the actual results of your program. Your results may vary due to random shuffling, hyperparameter choices, and other implementation details.

Why log scale for C? Because you sweep across orders of magnitude; log scale makes multiplicative steps (×10) equally spaced and readable.
What happens as C increases/decreases?
Larger C = stronger penalty for violations → tends to fit training data harder, often smaller margin (larger $\|w\|$), can overfit. Notice when C=1000 test accuracy went down. But in general, increasing C increases test accuracy, as expected.
Smaller C = allows more violations → often larger margin (smaller $\|w\|$), can underfit.
The plots should show this tradeoff: accuracy vs margin changes with C.



Margin vs C (regularisation parameter)



Accuracy vs C (regularisation parameter)